Long-Context Modeling with Transformer Variants

Abhinav Saraf

Abstract

Long-context modeling remains a core challenge in modern sequence models due to the quadratic complexity of vanilla self-attention and limitations in positional extrapolation. This study investigates architectural approaches for improving long-context modeling in decoder-only Transformers, focusing on efficient attention mechanisms, positional encoding methods, and convolution-attention hybrid architectures. We evaluate these approaches across multiple context lengths with respect to model perplexity, throughput and extrapolation capability, and analyze the trade-offs introduced by each design choice.

1 Introduction

Transformers(Vaswani et al., 2017) have become the dominant architecture for sequence modeling, but their quadratic attention complexity and limited positional generalization pose challenges for long-context tasks. In this work, we explore multiple architectural modifications to standard self attention to improve efficiency and scalability such as Grouped Query Attention(Ainslie et al., 2023), Sliding Window Local Attention(Beltagy et al., 2020) and Linear Attention(Katharopoulos et al., 2020). We also experiment with different positional embeddings such as RoPE(Su et al., 2024), ALiBi(Press et al., 2021) and Relative Positional Encoding(Shaw et al., 2018). Additionally, we also modify the architecture to include a 1D convolutional component to efficiently capture local n-gram context(Gulati et al., 2020). All experiments are carried out on WikiText-2(Merity et al., 2016), with the task being next word prediction on different context lengths.

2 Baseline Transformer

We implement a standard Transformer model as described in (Vaswani et al., 2017). We use the GPT-2(Radford et al., 2019) Tokenizer which has $vocabsize = 50257$ to tokenize the dataset. The input tokens pass through an embedding layer which is a learnable lookup table converting each token into a embedding. Sinusoidal positional encodings are added and the resulting embeddings are passed through a stack of transformer layers.

Every transformer block consists of a masked self attention block and a feed forward neural network. In the self attention block, the upper triangle of attention weights are set to $-\infty$ which ensures that a word is not able to attend to words in front of it. We use the Post-LN approach in which layer norms come after the blocks, after which there are residual connections. The final output layer maps the embeddings to tokens and hence uses the same weights as the initial embedding layer.

To establish a baseline, we use a decoder only Transformer model with 6 layers, using $d_{model} = 512$, $d_{ff} = 2048$, $h = 8$, $d_q = d_k = d_v = 64$. The baseline has 45 million parameters. It was trained for 3 epochs using AdamW(Loshchilov and Hutter, 2017), with $lr = 0.0005$, $betas = (0.9, 0.95)$,

weight_decay = 0.1. We use a cosine decay scheduler with warmup time as 10% of total steps. Each training batch consisted of 8 examples, each of which had a 1024-token sequence. We also used *accumulation_steps* = 4 to increase the effective batch size. The model was trained on Google Colab's T4 GPU.

Metric	Value
Train Loss	5.91
Val Loss	5.97
Perplexity	394.74
Throughput (tokens/sec)	16510.36
Peak GPU Memory (GB)	9.74
Time per epoch (seconds)	143.25

Table 1: Baseline performance

3 Attention Variants

We evaluate multiple attention mechanisms such as GQA, Linear and Sliding Window attention over varying context lengths.

3.1 Grouped Query Attention

GQA(Ainslie et al., 2023) divides query heads into G groups, each of which shares a single key and value head. $G = 1$ is equivalent to Multi Query Attention(Shazeer, 2019), while $G = H$ is the same as Multi-Head attention. MQA speeds up decoder inference by reducing the size of the KV cache by a factor of $H(heads)$, lowering memory requirements during autoregressive generation. However, using shared keys and values can lead to degradation in model quality. As a compromise, GQA introduces groups, retaining more representational diversity than MQA while still achieving faster decoder inference than Multi-Head attention.

3.2 Sliding Window Attention

Sliding window attention(Beltagy et al., 2020) restricts each query token to attend only to a fixed window of w previous tokens, rather than the entire sequence. This reduces the computational complexity of attention from $O(n^2)$ to $O(n \cdot w)$, making it more efficient for long sequences. However, this constraint limits the model's ability to capture long-range dependencies, as tokens cannot directly attend to positions outside their window. Although this restricts direct interactions, information can still propagate across tokens over multiple layers, resulting in an effective receptive field that grows with depth, similar to CNNs.

It provides a trade-off between efficiency and context: smaller window sizes improve computational efficiency but restrict context, while larger windows better approximate full attention at increased cost. Unlike GQA, sliding window attention reduces time rather than memory, as the full KV cache is still maintained during autoregressive inference.

3.3 Linear Attention

A major bottleneck in Multi-Head Attention is the $n \times n$ attention matrix, which causes $O(n^2)$ time and memory complexity. Linear attention(Katharopoulos et al., 2020) addresses this by reformulating the attention computation to avoid explicitly computing this matrix.

Instead of applying the softmax function, linear attention uses a kernel feature map $\phi(\cdot)$ to transform queries and keys, allowing the attention operation to be rewritten by associativity as:

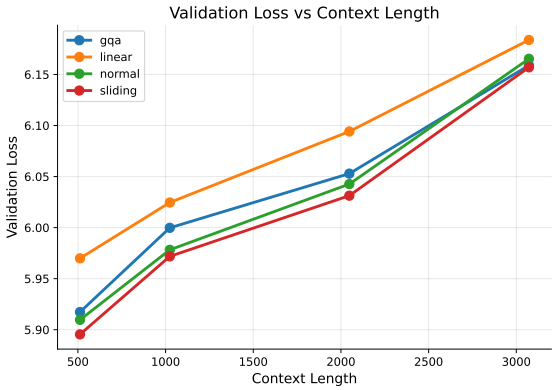
$$\text{Attn}(Q, K, V) = \phi(Q) \left(\phi(K)^T V\right)$$

This enables computing the intermediate term $\phi(K)^T V$ once, resulting in a $d \times d$ matrix that can be reused across all queries. As a result, both time and memory complexity scale linearly with sequence length, i.e., $O(n)$, making linear attention particularly effective for long-context settings.

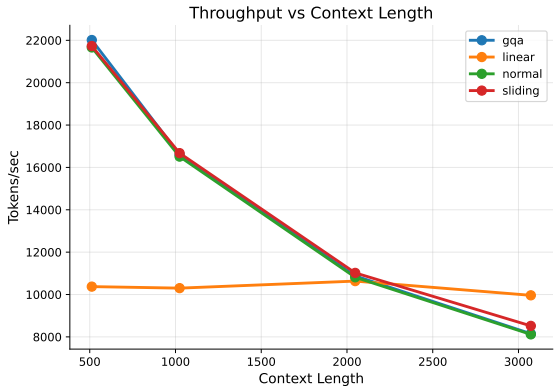
3.4 Experiments

Context	Attention	Train Loss	Val Loss	Perplexity	Throughput (tok/s)	Memory (GB)	Time/Epoch (s)
512	Normal	5.89	5.90	368.58	21663.69	8.30	108.31
	GQA	5.90	5.91	371.39	22014.27	8.34	106.59
	Linear	5.94	5.96	391.46	10372.82	10.76	226.21
	Sliding	5.89	5.89	363.36	21731.83	8.29	107.97
1024	Normal	5.91	5.97	394.74	16510.36	9.74	143.25
	GQA	5.70	5.99	403.34	16597.66	9.76	142.50
	Linear	5.97	6.02	413.47	10301.31	10.77	229.60
	Sliding	5.80	5.97	392.21	16674.55	9.72	141.84
2048	Normal	6.07	6.04	421.00	10813.41	12.66	217.70
	GQA	6.06	6.05	425.34	10878.92	12.61	216.38
	Linear	5.86	6.09	443.27	10633.32	10.78	221.38
	Sliding	5.93	6.03	416.25	11020.58	12.56	213.60
3072	Normal	6.22	6.16	475.97	8117.72	12.02	212.61
	GQA	6.17	6.15	472.91	8150.48	11.82	216.79
	Linear	6.02	6.18	484.82	9962.00	8.30	176.85
	Sliding	6.11	6.15	472.01	8519.29	11.79	211.58

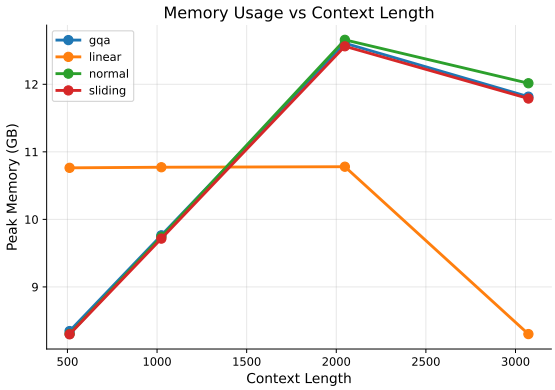
Table 2: Comparison of attention mechanisms across different context lengths. The number of optimizer steps is fixed at 870 across experiments, with $G = 2$ and sliding window size $w = 512$. To maintain a constant token budget across runs, batch size is scaled as $batch\_size = K/\text{context}$ where $K = 8192$, except for context length 3072 where a batch size of 2 is used due to memory constraints.



(a) Validation loss vs context length



(b) Throughput vs context length



(c) Memory usage vs context length

(d) Speed-quality tradeoff

Figure 1: Comparison of attention mechanisms across validation performance, throughput, memory usage, and speed-quality tradeoffs.

The 3072-context experiments used 6144 tokens per batch rather than 8192 due to memory constraints, resulting in 25% lower token exposure per optimization step. As such, results at 3072 may slightly underestimate achievable performance and due to this, the memory and epoch time is slightly less than what it would have been.

Although marginally, Sliding Window has the lowest perplexity for all context lengths. This may be because most of the tokens relevant for a given token lie within its local window. Linear attention has a near constant throughput which causes it to be much more efficient at higher context lengths such as 3072, but this comes at the cost of performance, which is worse than the other variants. This is because of the approximations used instead of softmax. We also observe that increasing context length worsens performance, since modeling longer sequences may need more training to converge.

4 Positional Embeddings

In (Vaswani et al., 2017), sinusoidal positional embeddings are added to the semantic embeddings, resulting in transformer models memorizing rather than generalizing positions. This causes performance to deteriorate when the context length exceeds the training data. We compare the performance of methods such as RoPE, ALibi and relative positional encoding which try to solve this problem.

4.1 Rotary Positional Embedding

RoPE (Su et al., 2024) encodes positional information by applying a rotation to the query and key vectors in every transformer layer. This rotation preserves vector norms while enabling attention scores to depend on relative positions between tokens. The rotation angle is determined by the token position m and the dimension index i. Lower dimensions (small i) have higher angular frequencies, resulting in rapid rotations that capture local dependencies, while higher dimensions rotate more slowly, encoding long-range relationships. This allows the model to represent both short- and long-range interactions across different dimensions of the embedding.

$$\text{angle} = m \cdot \theta$$
$$\theta = 10000^{-\frac{2(i-1)}{d_{\text{model}}}}$$

Rotation is applied to each pair of dimensions using the standard 2D rotation matrix.

$$\begin{pmatrix} x'_{2i} \\ x'_{2i+1} \end{pmatrix} = \begin{pmatrix} \cos(m\theta_i) & -\sin(m\theta_i) \\ \sin(m\theta_i) & \cos(m\theta_i) \end{pmatrix} \begin{pmatrix} x_{2i} \\ x_{2i+1} \end{pmatrix}$$

4.2 Attention with Linear Biases

ALiBi (Press et al., 2021) incorporates positional information by adding a fixed linear bias directly to the attention scores, rather than modifying token embeddings. For each attention head h, a predefined slope m_h is assigned, and a bias proportional to the relative distance between query position i and key position j is added:

$$B_h(i, j) = m_h(j - i)$$

The attention is then modified as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + B_h\right)V$$

where B_h penalizes attention to distant tokens, with larger negative biases assigned to tokens further in the past. Each head uses a different slope, allowing different heads to represent global and local dependencies. Unlike learned or explicit positional embeddings, ALiBi introduces no additional trainable parameters and allows transformers trained on shorter context lengths to generalize more effectively to longer sequences at inference.

4.3 Relative Positional Encoding

Relative positional encoding (Shaw et al., 2018) represents token positions based on their relative distance rather than absolute position. For a query at position i attending to a key at position j, a learnable embedding R_{i-j} is assigned based on the relative offset $(i - j)$:

$$R_{i-j} \in \mathbb{R}^{d_k}$$

where relative distances are clipped to a fixed maximum range to limit the number of learned embeddings. These embeddings are added to the attention logits:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T + QR^T}{\sqrt{d_k}}\right)V$$

where QR^T represents the contribution of relative positional information. Unlike absolute positional embeddings, this method allows attention scores to depend on token distances rather than fixed positions.

4.4 Extrapolation Test

Positional Encoding	Eval-512	Eval-1024	Eval-2048	ΔPerplexity (%)
Sinusoidal	368.58	414.78	467.18	26.7
RoPE	270.47	273.94	285.17	5.4
ALiBi	272.29	271.03	271.11	-0.4
Relative	339.04	343.73	352.51	4.0

Table 3: Context length extrapolation results in terms of perplexity for positional encoding methods trained at sequence length 512 and evaluated at sequence lengths 512, 1024, and 2048. Normalized degradation is computed as the percentage change in perplexity at 2048 relative to 512.

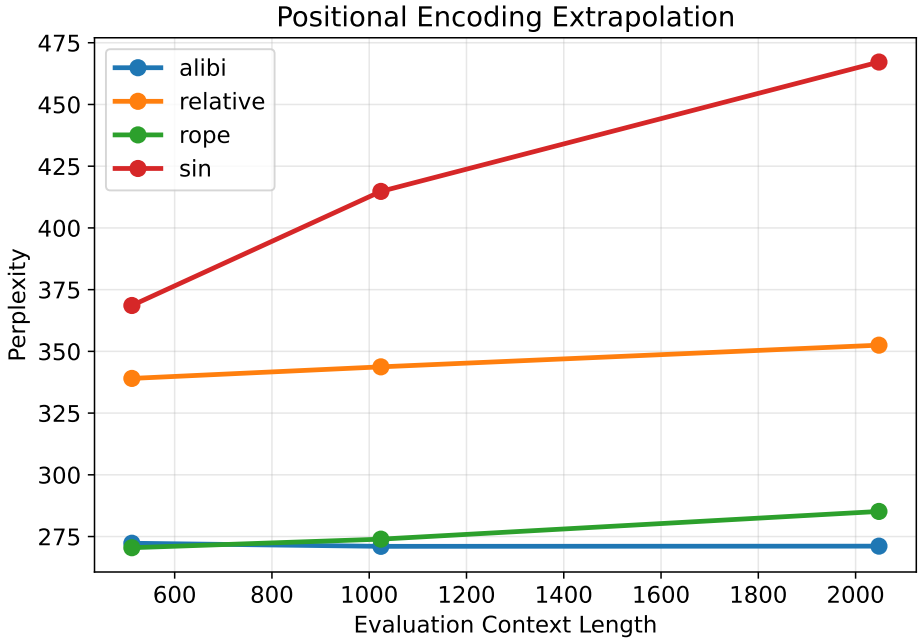


Figure 2: Perplexity as a function of evaluation context length for positional encoding methods trained at sequence length 512. ALiBi shows near-constant performance under extrapolation, because its distance-based penalty adjustment extends naturally beyond the training sequence length. RoPE and relative positional embeddings only show mild degradation, with the relative embeddings being helped by distance based clipping. Sinusoidal positional encoding degrades substantially at longer contexts because the model is evaluated on positional patterns beyond those seen during training, and it is unable to generalize.

5 Convolution and Attention Hybrid Architectures

To improve the modelling of local dependencies while retaining the global context modelling capabilities of self-attention, two convolution-attention hybrid architectures were investigated.

5.1 Convolution Augmented Attention

The first hybrid design places a 1D convolutional layer before the self-attention block. Rather than relying solely on attention at every layer, convolutions are also introduced to capture local dependencies (Gulati et al., 2020).

5.2 Gated Convolutional Feedforward Networks

The second hybrid design modifies the standard Transformer feedforward network by replacing the FFN with a gated convolutional feedforward network (Dauphin et al., 2017). This module combines linear projections, gating, and causal convolution to improve local feature extraction within the feedforward sublayer.

Given input x, the module computes two projections:

$$H = W_h x$$

$$G = W_g x$$

where H represents the information path and G represents the gating path. A causal convolution is applied to the gating branch, followed by an element-wise gating operation:

$$Y = H \odot GeLU(\text{Conv}(G))$$

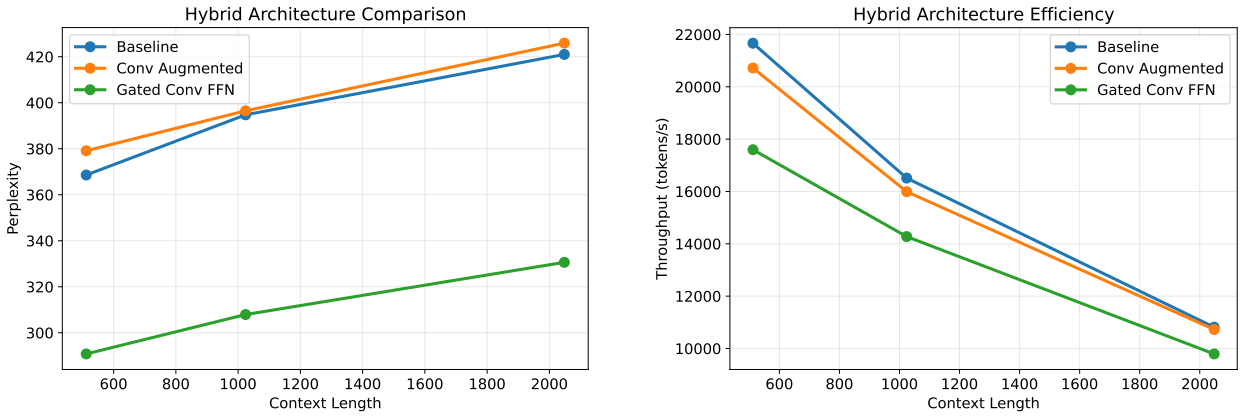
where $\odot$ denotes element-wise multiplication.

By introducing convolution into the feedforward pathway, the model can capture local contextual patterns without modifying the attention mechanism directly.

5.3 Experiments

Context	Attention	Train Loss	Val Loss	Perplexity	Throughput (tok/s)	Memory (GB)	Time/Epoch (s)
512	Normal	5.89	5.90	368.58	21663.69	8.30	108.31
	Conv Augmented	5.92	5.93	379.12	20716.74	8.48	113.27
	Gated Conv FFN	5.48	5.67	290.80	17595.69	8.95	133.36
1024	Normal	5.91	5.97	394.74	16510.36	9.74	143.25
	Conv Augmented	6.02	5.98	396.49	15991.59	9.92	147.90
	Gated Conv FFN	5.68	5.72	307.92	14275.64	10.39	165.68
2048	Normal	6.07	6.04	421.00	10813.41	12.66	217.70
	Conv Augmented	6.14	6.05	425.91	10725.73	12.84	219.48
	Gated Conv FFN	5.91	5.80	330.62	9791.29	13.31	240.42

Table 4: We evaluate the hybrid architectures on varying context lengths using Multi-Head attention and sinusoidal positional embeddings.



(a) Perplexity versus context length.

(b) Throughput versus context length.

Figure 3: Performance and efficiency trade-offs for hybrid architectures across varying context lengths. The Gated Conv FFN achieves substantially lower perplexity than the baseline and Conv-Augmented model, at the cost of reduced throughput. The gated FFN combines local feature extraction with dynamic feature selection via gating which improves performance.

6 Best Combined Model

We combine the best-performing components from previous sections: Sliding Window attention, Rotary positional embeddings and a Gated Convolutional FFN. We compare a model with these modifications to the initial baseline. Perplexity goes down by 32.2% while throughput is only reduced by 13.6%, which represents a favourable tradeoff.

Context	Model	Train Loss	Val Loss	Perplexity	Throughput (tok/s)	Memory (GB)	Time/Epoch (s)
1024	Baseline	5.91	5.97	394.74	16510.36	9.74	143.25
1024	Final	5.51	5.59	267.80	14265.25	10.37	165.80

Table 5: Final comparison

7 Limitations

Due to computational constraints, all experiments were conducted using a single NVIDIA T4 GPU under a fixed training budget of three epochs, which may limit convergence for some model variants. As a result, the reported results should be interpreted as comparative findings under a controlled compute budget rather than fully converged performance results.

8 Conclusion

This study evaluated multiple architectural approaches for improving long-context modeling in decoder-only Transformers, including efficient attention mechanisms, positional encoding methods, and convolution-attention hybrid architectures.

Among attention variants, Sliding Window attention achieved the strongest overall trade-off between perplexity, computational efficiency, and memory usage, while linear attention provided improved efficiency for long context lengths at the cost of reduced performance.

For positional encodings, relative methods consistently outperformed sinusoidal positional encoding under context extrapolation. In particular, ALiBi exhibited near-constant perplexity when evaluated beyond its training context, while RoPE and relative positional encoding showed only mild degradation, substantially outperforming sinusoidal embeddings.

Among convolution-attention hybrids, the Gated Conv FFN produced the largest performance gains, significantly reducing perplexity relative to the baseline across all tested context lengths, while simple convolution augmentation before attention provided no such benefit. Combining the best-performing components: Sliding Window attention, RoPE, and the Gated Conv FFN yielded the strongest overall model.

Overall, the results suggest that improvements to positional encoding and feedforward sublayers can provide substantial gains for long-context modeling, which can be much more than those obtained from modifying attention mechanisms alone.

References

Joshua Ainslie, James Lee-Thorp, Michiel De Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 4895–4901, 2023.

Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

Yann N Dauphin, Angela Fan, Michael Auli, and David Grangier. Language modeling with gated convolutional networks. In *International conference on machine learning*, pages 933–941. PMLR, 2017.

Anmol Gulati, James Qin, Chung-Cheng Chiu, Niki Parmar, Yu Zhang, Jiahui Yu, Wei Han, Shibo Wang, Zhengdong Zhang, Yonghui Wu, et al. Conformer: Convolution-augmented transformer for speech recognition. *arXiv preprint arXiv:2005.08100*, 2020.

Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are rnns: Fast autoregressive transformers with linear attention. In *International conference on machine learning*, pages 5156–5165. PMLR, 2020.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.

Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843*, 2016.

Ofir Press, Noah A Smith, and Mike Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. *arXiv preprint arXiv:2108.12409*, 2021.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.

Peter Shaw, Jakob Uszkoreit, and Ashish Vaswani. Self-attention with relative position representations. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*, pages 464–468, 2018.

Noam Shazeer. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.

Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.